```
import pandas as pd
from sklearn.datasets import load_iris

iris_data = load_iris()
X = pd.DataFrame(iris_data.data, columns=iris_data.feature_names)
y = pd.Series(iris_data.target, name="target")
df = pd.concat([X, y], axis=1)
print(df.head())
```

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   target
0       0
1       0
2       0
3       0
4       0
```

```
import matplotlib.pyplot as plt
import seaborn as sns

print(df.info())
print(df.shape)
print(df.describe())

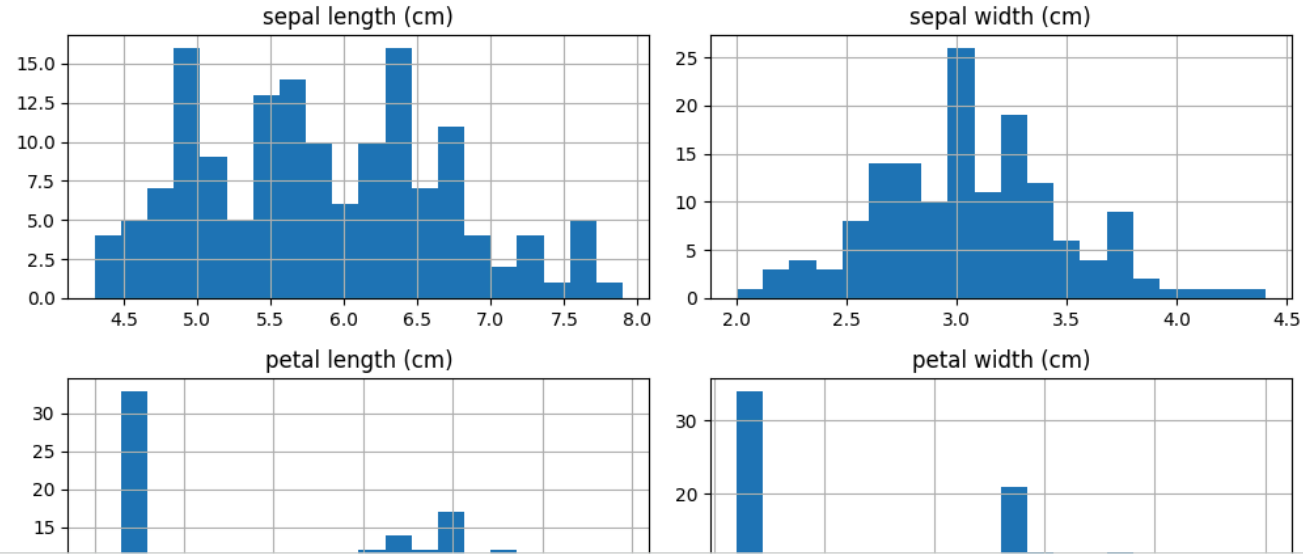
df.hist(figsize=(10, 8), bins=20)
plt.tight_layout()
plt.show()

sns.pairplot(df, hue="target", palette="husl")
plt.show()

plt.figure(figsize=(8, 6))
sns.heatmap(df.corr(), annot=True, cmap="magma", fmt=".2f")
plt.title("Correlation Matrix")
plt.show()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   target             150 non-null    int64  
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
None
(150, 5)
       sepal length (cm)  sepal width (cm)  petal length (cm)  \
count         150.000000        150.000000         150.000000   
mean            5.843333          3.057333           3.758000   
std             0.828066          0.435866           1.765298   
min             4.300000          2.000000           1.000000   
25%             5.100000          2.800000           1.600000   
50%             5.800000          3.000000           4.350000   
75%             6.400000          3.300000           5.100000   
max             7.900000          4.400000           6.900000   

       petal width (cm)      target  
count        150.000000  150.000000  
mean           1.199333    1.000000  
std            0.762238    0.819232  
min            0.100000    0.000000  
25%            0.300000    0.000000  
50%            1.300000    1.000000  
75%            1.800000    2.000000  
max            2.500000    2.000000  
```




```
from sklearn.model_selection import train_test_split

features = df.drop("target", axis=1)
labels = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    features, labels, test_size=0.30, random_state=101
)
print(X_train.shape, X_test.shape)
```

```
(105, 4) (45, 4)
```

```
from sklearn.tree import DecisionTreeClassifier

dt_model = DecisionTreeClassifier(random_state=101)
dt_model.fit(X_train, y_train)
```

```
DecisionTreeClassifier(random_state=101)
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

predictions = dt_model.predict(X_test)

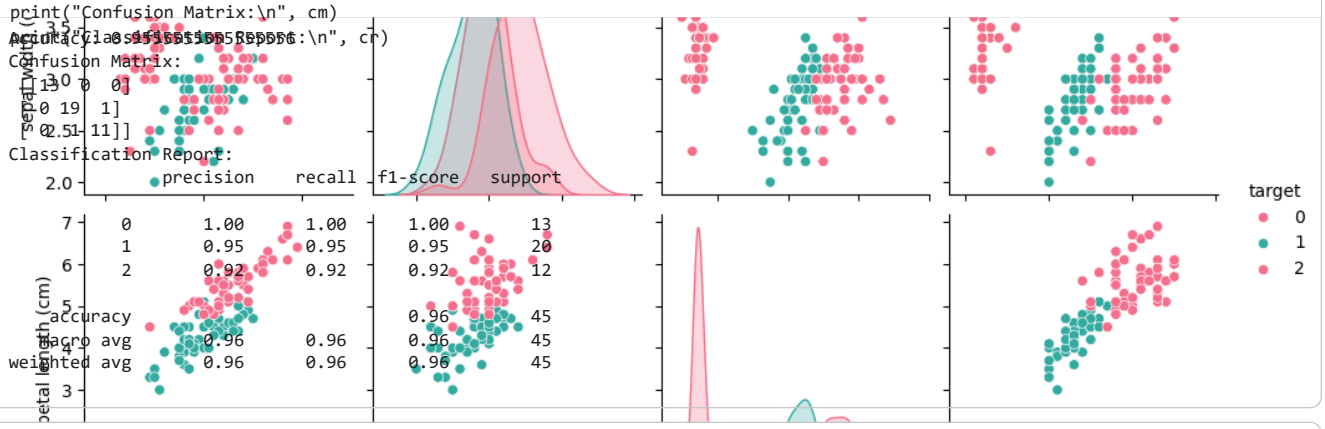
acc = accuracy_score(y_test, predictions)
cm = confusion_matrix(y_test, predictions)
cr = classification_report(y_test, predictions)

print("Accuracy:", acc)
print("Confusion Matrix:\n", cm)
print("Classification Report:\n", cr)
```

```
Accuracy: 0.9555555555555556
Confusion Matrix:
 [[13  0  0]
 [ 0 19  1]
 [ 0  1 11]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        13
           1       0.95      0.95      0.95        20
           2       0.92      0.92      0.92        12

    accuracy                           0.96        45
   macro avg       0.96      0.96      0.96        45
weighted avg       0.96      0.96      0.96        45
```




```
import numpy as np

criteria = ["gini", "entropy"]
depths = [None, 2, 4, 6]
splits = [2, 5, 10]

for c in criteria:
    for d in depths:
        for s in splits:
            exp_model = DecisionTreeClassifier(
                criterion=c, max_depth=d, min_samples_split=s, random_state=101
            )
            exp_model.fit(X_train, y_train)

            train_score = exp_model.score(X_train, y_train)
            test_score = exp_model.score(X_test, y_test)

            status = ""
            if train_score - test_score > 0.05:
                status = "Overfitting"
            elif train_score < 0.85 and test_score < 0.85:
                status = "Underfitting"
            else:
                status = "Good Fit"

            print(f"Crit:{c} | Depth:{d} | Split:{s} | Train:{train_score:.3f} | Test:{test_score:.3f} | {status}")
```

```
Crit:gini | Depth:None | Split:2 | Train:1.000 | Test:0.956 | Good Fit
Crit:gini | Depth:None | Split:5 | Train:0.981 | Test:0.978 | Good Fit
Crit:gini | Depth:None | Split:10 | Train:0.981 | Test:0.978 | Good Fit
Crit:gini | Depth:2 | Split:2 | Train:0.971 | Test:0.933 | Good Fit
Crit:gini | Depth:2 | Split:5 | Train:0.971 | Test:0.933 | Good Fit
Crit:gini | Depth:2 | Split:10 | Train:0.971 | Test:0.933 | Good Fit
Crit:gini | Depth:4 | Split:2 | Train:1.000 | Test:0.956 | Good Fit
Crit:gini | Depth:4 | Split:5 | Train:0.981 | Test:0.978 | Good Fit
Crit:gini | Depth:4 | Split:10 | Train:0.981 | Test:0.978 | Good Fit
Crit:gini | Depth:6 | Split:2 | Train:1.000 | Test:0.956 | Good Fit
Crit:gini | Depth:6 | Split:5 | Train:0.981 | Test:0.978 | Good Fit
Crit:gini | Depth:6 | Split:10 | Train:0.981 | Test:0.978 | Good Fit
```

